

PROJECT AND TEAM INFORMATION

Project Title

AI-powered Mood-Based Music

Student / Team Information

<i>Team Name:</i> <i>Team :</i>	STAT-IV-T018
Team member 1 (Team Lead)	<i>Tripathi, Shorya – 230213752</i> <i>Shoryatripathi0606@gmail.com</i> 
Team member 2	<i>Tushar – 23021717</i> <i>23021717@geu.ac.in</i> 
Team member 3	<i>Kumari, Sagun – 23012356</i> <i>shmehta109@gmail.com</i> 

PROPOSAL DESCRIPTION (10 pts)

- **Objective:** To develop a smart music recommendation system that suggests songs based on the user's current mood using AI and statistical analysis.
- **User Input:** Users can either select their mood manually (e.g., happy, sad, energetic) or input text (like tweets or diary entries) which will be analyzed to detect mood.
- **Dataset Utilization:** Uses audio feature datasets from sources like Spotify API or Kaggle's Million Song Dataset, extracting attributes like tempo, danceability, and energy.
- **Statistical Analysis:** Performs correlation analysis to understand how different song features align with various moods.
- **Sentiment Detection:** Implements NLP models (like TF-IDF + Naive Bayes, or BERT) to determine mood from user text input.
- **AI Model:** Trains machine learning algorithms such as K-Nearest Neighbors, Decision Trees, or Neural Networks to map moods to songs.
- **Recommendation Engine:** Matches the detected mood with a suitable list of songs using the trained AI model and audio feature analysis.
- **User Interface:** A simple and intuitive UI built using Flask or Streamlit allows users to interact with the system easily.
- **Project Timeline:** Completed in three weeks—Week 1 (Data & Mood Processing), Week 2 (Modeling), Week 3 (UI and Testing).
- **Outcome:** A functional prototype that recommends mood-based songs, showcasing an effective integration of AI, sentiment analysis, and statistical modeling.

Motivation

Most music platforms recommend songs based on user history or popularity, ignoring the listener's current emotional state. However, mood plays a significant role in how we connect with music. This project aims to bridge that gap by creating a system that recommends songs based on the user's mood—either selected manually or detected through text input. By combining sentiment analysis and machine learning, the system offers a more personalized and emotionally aware listening experience, making music recommendations more meaningful and context-sensitive.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

Currently, most music streaming platforms like Spotify, Apple Music, and YouTube Music use collaborative filtering and content-based filtering to recommend songs. These systems rely heavily on user listening history, likes, and playlists to suggest similar tracks. Some platforms offer mood-based playlists, but these are manually curated and do not adapt in real time to a user's current emotional state. Few applications have integrated sentiment analysis or real-time mood detection. While some experimental apps and research projects attempt emotion-based music recommendation, they often lack scalability, personalization, or use limited mood detection techniques. Additionally, these solutions rarely combine both statistical audio feature analysis and AI-based mood classification from user input. Thus, there is a gap in dynamic, intelligent, and personalized mood-based music recommendation systems.

Project Goals and Milestones (2 pts)

The main goal of this project is to develop a mood-aware music recommendation system that uses user mood input—either selected directly or extracted from text—to suggest appropriate songs. The system aims to blend statistical analysis of song features with AI-driven mood detection and recommendation to provide a more personalized and emotionally relevant music experience.

Milestones:

- **Week 1: Data Collection & Preprocessing**
 - Gather datasets from Spotify API and Kaggle.
 - Extract and clean audio features such as tempo, energy, and danceability.
 - Define mood categories and label initial dataset accordingly.
- **Week 2: Sentiment & Statistical Analysis**
 - Implement NLP techniques (TF-IDF, Naive Bayes, or BERT) for mood detection from text input.
 - Perform correlation analysis between song features and mood categories.
- **Week 3: Model Training & UI Development**
 - Train machine learning models (KNN, Decision Tree, or Neural Networks) to recommend songs based on detected mood.
 - Build a simple, interactive web interface using Flask or Streamlit.
 - Test and fine-tune the model using user feedback and accuracy metrics.
- **Final Milestone:**
 - Deliver a working prototype that allows users to input their mood and receive personalized song suggestions, with optional text-based mood detection.

Project Approach

- **Data Collection:** Use Spotify API and Kaggle's Million Song Dataset to extract audio features (tempo, energy, valence, danceability, etc.).
- **Mood Input Options:**
 - Users can either select a mood manually (happy, sad, relaxed, etc.).
 - Or enter text expressing their emotions (e.g., a tweet or diary entry).
- **Mood Detection via NLP:**
 - Use techniques like TF-IDF with Naive Bayes or BERT to classify mood from text input.
- **Statistical Analysis:**
 - Perform correlation analysis between audio features and mood categories to understand patterns.
- **Machine Learning Model:**
 - Train models like K-Nearest Neighbors, Decision Trees, or Neural Networks for song recommendations.
- **Platform & Tools:**
 - Python for backend development.
 - Libraries: Scikit-learn, Pandas, NLTK, TensorFlow.
 - Flask or Streamlit for building the web interface.
- **Testing & Refinement:**
 - Use feedback and accuracy metrics to iteratively improve recommendations.

System Architecture (High Level Diagram)(2 pts)

- **User Input:**

Users provide either a mood selection (e.g., happy, sad) or enter text expressing their feelings.
- **Mood Detection Module:**

Uses NLP techniques (TF-IDF, Naive Bayes, BERT) to analyze text and determine the user's emotional state.
- **Feature Extraction:**

Retrieves audio features from the song dataset (tempo, energy, danceability, etc.).

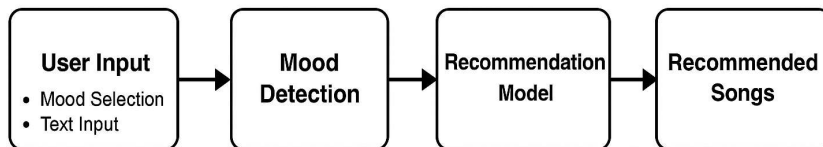
- **Recommendation Engine:**

Machine Learning model (KNN, Decision Tree, etc.) maps the detected mood to suitable songs using feature correlations.

- **Output Interface:**

A simple web UI (Flask or Streamlit) displays a list of recommended songs tailored to the user's mood.

Mood-Based Music Recommendation



Project Outcome / Deliverables (1 pts)

The primary outcome of this project is a functional prototype of a **Mood-Based Music Recommendation System** that can suggest songs aligned with the user's emotional state. The deliverables include:

- **Mood Detection Module:** A system that can identify user mood from text input using NLP and sentiment analysis.
- **Audio Feature-Based Recommendation Engine:** A trained machine learning model that matches user moods with song features (e.g., tempo, valence, energy) to provide suitable song recommendations.
- **Interactive Web Interface:** A user-friendly frontend (built using Flask or Streamlit) where users can select a mood or input text to receive music suggestions.
- **Documentation:** Detailed documentation covering system architecture, dataset description, algorithms used, and implementation steps.
- **Evaluation Report:** A brief performance summary including accuracy metrics of the mood detection model and feedback from user testing.

Together, these deliverables demonstrate the integration of sentiment analysis, statistics, and AI to provide a smart and emotionally responsive music recommendation system.

Assumptions

- Users can accurately express their mood through either manual selection or short text input.
- The available audio features (e.g., tempo, energy, valence) are sufficient to represent the emotional tone of a song.
- The mood categories (e.g., happy, sad, calm, energetic) are generalizable and applicable across a broad user base.
- Pre-trained NLP models and public datasets are reliable enough for mood detection and music classification.
- Users have access to an internet connection and a browser to interact with the web interface.

References

- Spotify Web API Documentation – <https://developer.spotify.com/documentation/web-api>
- Million Song Dataset – <https://labrosa.ee.columbia.edu/millionsong>
- Scikit-learn Machine Learning Library – <https://scikit-learn.org>
- NLTK Natural Language Toolkit – <https://www.nltk.org>
- BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding – <https://arxiv.org/abs/1810.04805>
- Flask Web Framework – <https://flask.palletsprojects.com>
- Streamlit for Interactive Web Apps – <https://streamlit.io>
- Kaggle Datasets – <https://www.kaggle.com>